

AI Аналитика в LoviEat - Техническая документация

Обзор

AI аналитика в приложении LoviEat находится на вкладке "Статистика" → "AI Аналитика". Система анализирует данные о питании пользователя за последние 30 дней и предоставляет персонализированные рекомендации.

Архитектура системы

Основные компоненты

- Stats.tsx** (src/pages/Stats.tsx) - UI компонент страницы статистики
- nutritionAnalytics.ts** (src/lib/nutritionAnalytics.ts) - аналитический движок
- firestore.ts** - загрузка данных из Firebase
- storage.ts** - локальное хранение норм КБЖУ

Поток данных

```
Firestore (diary, weight, activity)
  ↓
Stats.tsx (загрузка данных за 30 дней)
  ↓
nutritionAnalytics.ts (анализ)
  ↓
NutritionAnalyticsResult (результаты)
  ↓
UI отображение (11 блоков)
```

Блоки AI аналитики

1. AI Оценка питания (Overall Score)

За что отвечает:

- Общая оценка качества питания пользователя от 0 до 100
- Цветовая индикация: зелёный (90+), лайм (75+), жёлтый (60+), оранжевый (40+), красный (<40)

Используемые данные:

- proteinCompliance.score - выполнение белковой нормы
- stability.stabilityScore - стабильность калорий
- deficitAnalysis.discrepancy - расхождение ожидаемой и фактической потери веса
- streaks.calorieStreak - серия дней с нормой калорий
- streaks.proteinStreak - серия дней с нормой белка
- recovery.riskLevel - риск восстановления

Взаимодействие с другими блоками:

- Агрегирует результаты из 5 других блоков
- Используется для быстрой оценки общего состояния

Формула расчёта:

```
Базовый счёт: 50
+ proteinCompliance.score * 0.25 (макс +25)
+ stability.stabilityScore * 0.20 (макс +20)
+ deficit точность (макс +15)
+ streaks бонус (макс +20)
- recovery штраф (высокий: -15, средний: -8, низкий: -3)
```

2. Сводка дня (Daily Verdict)

За что отвечает:

- Персонализированное сообщение о текущем дне

- Рекомендации на основе сегодняшнего питания

Используемые данные:

- foodLogsByDay - записи питания по дням
- dailyDeficit - ежедневный дефицит
- targetCalories - целевые калории
- targetProtein - целевой белок
- plateau.plateau - наличие плато
- proteinCompliance - выполнение белковой нормы

Взаимодействие с другими блоками:

- Использует данные из блока Plateau Detection
- Использует результаты Protein Compliance
- Независимый блок для отображения

Логика:

- Если нет записей сегодня → рекомендация начать день
- Идеальный день (калории 90-105%, белок $\geq 90\%$) → похвала
- Калории в норме, белок низкий → предупреждение о голоде
- Калории низкие, белок хороший → предупреждение об энергии
- Превышение калорий → рекомендация вернуться к норме

3. Выполнение белковой нормы (Protein Compliance)

За что отвечает:

- Оценка выполнения белковой нормы в процентах
- Количество успешных дней (белок $\geq 90\%$ от нормы)
- Средний недобор белка в неудачные дни
- Текстовый вердикт с рекомендациями

Используемые данные:

- foodLogsByDay - записи питания по дням
- targetProtein - целевой белок
- avgProtein - средний белок

Взаимодействие с другими блоками:

- Данные используются в Overall Score
- Данные используются в Daily Verdict
- Независимый расчёт

Формула:

score = (успешные дни / дни с данными) * 100
avgMiss = средний процент недобора в неудачные дни

Пороги оценки:

- 90%+ - Отлично
- 75%+ - Хорошо
- 60%+ - Нормально
- 40%+ - Требуется внимания
- <40% - Критично

4. Анализ дефицита (Deficit Analysis)

За что отвечает:

- Сравнение ожидаемой и фактической потери веса
- Определение расхождения в процентах
- Интерпретация причин расхождения

Используемые данные:

- dailyDeficit - ежедневный дефицит (BMR + активность - съедено)
- weightHistory - история веса
- avgCalories - средние калории

- `targetCalories` - целевые калории
- `foodLogsByDay` - записи питания по дням

Взаимодействие с другими блоками:

- Данные используются в Overall Score
- Данные используются в Recovery Risk
- Независимый расчёт

Формула:

Ожидаемая потеря = (сумма дефицита / 7700) * `adaptationFactor`

AdaptationFactor:

- <7 дней: 0.55
- 7-30 дней: 0.60
- 30-90 дней: 0.65
- >90 дней: 0.75

Фактическая потеря = вес в начале - вес в конце периода

Расхождение = |(фактическая - ожидаемая) / ожидаемая| * 100

Пороги расхождения:

- ≤10% - Отлично
- ≤35% - Нормальная вариативность
- ≤50% - Ожидаемый диапазон
- 50% - Проверить точность учёта

5. Обнаружение плато (Plateau Detection)

За что отвечает:

- Определение застоя в снижении веса
- Количество дней без прогресса
- Рекомендации при обнаружении плато

Используемые данные:

- `weightHistory` - история веса (последние 14 дней минимум)

Взаимодействие с другими блоками:

- Данные используются в Daily Verdict
- Отображается как предупреждение если обнаружено
- Независимый расчёт

Логика:

Если вес за 14 дней изменился менее чем на 0.5 кг:

```
plateau = true
daysStuck = 14
recommendation = "Снизить калории на 100-150 ккал, повысить активность"
```

6. Прогноз белка (Protein Forecast)

За что отвечает:

- Расчёт оставшегося белка на сегодня
- Срочность набора белка (высокая/средняя/низкая)
- Практические советы по продуктам

Используемые данные:

- `foodLogsByDay` - записи питания по дням
- `targetProtein` - целевой белок
- Текущее время (показывается только после 16:00)

Взаимодействие с другими блоками:

- Полностью независимый блок
- Показывается только если белок <70% от нормы

Логика:

```
Если текущий час <= 16: не показывать
Если сегодня нет записей: high urgency
Если ratio < 0.5: high urgency (срочно нужно Хг белка)
Если ratio < 0.7: medium urgency (осталось Хг белка)
Если ratio >= 0.7: не показывать
```

7. Стабильность калорий (Calorie Stability)

За что отвечает:

- Оценка стабильности потребления калорий
- Разброс (стандартное отклонение) в ккал
- Интерпретация влияния на прогресс

Используемые данные:

- foodLogsByDay - записи питания по дням

Взаимодействие с другими блоками:

- Данные используются в Overall Score
- Независимый расчёт

Формула:

```
variance = sqrt(Σ(calories - avg)² / n)
stabilityScore:
- variance < 200: 90 (отлично)
- variance < 500: 70 (хорошо)
- variance >= 500: 40 (средне)
```

8. Обнаруженные паттерны (Patterns)

За что отвечает:

- Выявление закономерностей в питании
- Переедание в выходные
- Недобор белка по будням
- Вечернее переедание
- Скачки калорий

Используемые данные:

- foodLogsByDay - записи питания по дням
- timestampsMeals - время приёмов пищи

Взаимодействие с другими блоками:

- Полностью независимый блок
- Информационный характер

Типы паттернов:

1. **Weekend** - выходные калории > будние на 20%+
2. **Weekday** - белок по будням < 80г
3. **Evening** - >40% калорий после 19:00
4. **Variance** - 2+ дней с резким превышением (>150% от среднего)

Уровни серьёзности:

- info - информационный
- warning - предупреждение
- alert - тревога

9. Риск восстановления (Recovery Risk)

За что отвечает:

- Оценка риска недостаточного восстановления
- Уровень риска (высокий/средний/низкий)
- Объяснение и рекомендации

Используемые данные:

- `dailyDeficit` - ежедневный дефицит
- `avgProtein` - средний белок
- `targetProtein` - целевой белок
- `activityCalories` - калории активности
- `weightHistory` - история веса
- `targetCalories` - целевые калории
- `foodLogsByDay` - записи питания по дням

Взаимодействие с другими блоками:

- Данные используются в Overall Score
- Показывается только при medium или high risk
- Независимый расчёт

Критерии высокого риска (все 4 должны быть выполнены):

- Дефицит >35% от TDEE
- Белок <70% от нормы
- Потеря веса >1% в неделю
- Продолжительность >7 дней

Уровни риска:

- 4 фактора: high
- 2-3 фактора: medium
- 0-1 фактор: low

10. Интерпретация веса (Weight Interpretation)

За что отвечает:

- Объяснение изменений веса
- Различение воды, флуктуаций, реального изменения
- Оценка темпа снижения

Используемые данные:

- `weightHistory` - история веса

Взаимодействие с другими блоками:

- Полностью независимый блок
- Показывается только если есть изменение

Логика:

```
Если изменение ±0.2 кг: stable (стабилен)
Если изменение >0:
  - <0.5 кг: fluctuation (нормальная флуктуация)
  - <1.0 кг: water (задержка воды)
  - >=1.0 кг: water (значительный прирост)
Если изменение <0:
  - >1% в неделю: fluctuation (быстрый темп)
  - >0.5% в неделю: fluctuation (безопасный темп)
  - иначе: fluctuation (комфортное снижение)
```

11. Серии (Streaks)

За что отвечает:

- Подсчёт последовательных успешных дней
- Три типа серий: калории, белок, дефицит
- Мотивационный элемент

Используемые данные:

- foodLogsByDay - записи питания по дням
- dailyDeficit - ежедневный дефицит
- targetCalories - целевые калории
- targetProtein - целевой белок

Взаимодействие с другими блоками:

- Данные используются в Overall Score
- Независимый расчёт

Определение успешного дня:

- **Calorie streak:** калории 90-110% от нормы
- **Protein streak:** белок $\geq 90\%$ от нормы
- **Deficit streak:** дефицит > 0

Форматирование:

- 0 дней: "Старт - Новая цель"
- 1 день: "1 день"
- 2-4 дня: "X дня"
- 5+ дней: "X дней"

Источники данных

Загрузка данных в Stats.tsx (строки 130-264)

```
// Загружаются данные за последние 30 дней:  
1. diaryEntries - записи из дневника питания (loadDiaryRange)  
2. weights - записи веса (loadWeight, 30 записей)  
3. activities - активность по дням (loadActivity, 30 дней)
```

Подготовка данных для аналитики

```
// Из diaryEntries формируется:  
foodLogsByDay = [{  
  date: string,  
  calories: number,  
  protein: number,  
  fat: number,  
  carbs: number,  
  entries: number  
}]  
  
// Рассчитывается:  
dailyDeficit = (BMR + activityCalories) - consumedCalories  
timestampsMeals = [{ hour, calories }]  
trackedDays = количество дней с записями  
streakDays = текущая серия отслеживания
```

Входные данные для analyzeNutrition

```
NutritionAnalyticsInput {  
  currentWeight: number  
  targetWeight?: number  
  avgCalories: number  
  avgProtein: number  
  avgFat: number  
  avgCarbs: number  
  dailyTargetCalories: number  
  dailyTargetProtein: number  
  dailyTargetFat: number  
  dailyTargetCarbs: number  
  activityCalories: number[]  
  dailyDeficit: number[]  
  weightHistory: { date, weight }[]  
  foodLogsByDay: [...]  
  trackedDays: number  
  streakDays: number  
  timestampsMeals: [...]  
}
```

Взаимодействия между блоками

Диаграмма зависимостей

```
Overall Score  
├─ Protein Compliance  
├─ Stability  
├─ Deficit Analysis  
├─ Streaks  
└─ Recovery Risk  
    └─ Deficit Analysis (данные)  
  
Daily Verdict  
├─ Plateau Detection  
└─ Protein Compliance  
  
Recovery Risk  
├─ Deficit Analysis (логика похожая)  
└─ использует те же входные данные
```

Общие входные данные

Все блоки используют следующие общие данные:

- foodLogsByDay - используется в 8 из 11 блоков
- weightHistory - используется в 4 блоках
- dailyDeficit - используется в 3 блоках
- Целевые значения (calories, protein) - используются в 6 блоках

Независимые блоки

Следующие блоки не зависят от других:

- Plateau Detection
- Protein Forecast
- Stability
- Patterns
- Weight Interpretation
- Streaks

Константы и пороги

nutritionAnalytics.ts

```
CALORIES_PER_KG = 7700 // калорий в 1 кг жира
PROTEIN_THRESHOLD_LOW = 0.7 // 70% от нормы
PROTEIN_THRESHOLD_HIGH = 0.9 // 90% от нормы
DEFICIT_THRESHOLD_LOW = 300 // ккал
DEFICIT_THRESHOLD_HIGH = 800 // ккал
PLATEAU_DAYS = 14 // дней для определения плато
VARIANCE_THRESHOLD_LOW = 200 // ккал
VARIANCE_THRESHOLD_HIGH = 500 // ккал
```

Фильтрация данных

Важное замечание о датах

Некоторые функции фильтруют данные начиная с 2026-05-12:

- analyzeDeficit - строки 186-195
- assessRecoveryRisk - строки 532-536
- calculateStreaks - строки 647-648
- generateDailyVerdict - строки 736-743

Это сделано для исключения старых/некорректных данных из анализа.

Производительность

Когда запускается расчёт

```
useEffect(() => {
  calculateAnalytics();
}, [entries, norm, user, weightEntries]);
```

Пересчёт происходит при изменении:

- Записей питания
- Нормы КБЖУ
- Пользователя
- Записей веса

Оптимизация

- Данные загружаются параллельно (Promise.all)
- Расчёт происходит только если есть данные
- Фильтрация по датам уменьшает объём обработки

UI отображение

Цветовая кодировка

Overall Score:

- 90+ : зелёный
- 75+ : лайм
- 60+ : жёлтый
- 40+ : оранжевый
- <40 : красный

Deficit Analysis:

- Расхождение $\leq 20\%$: зелёный
- Расхождение $> 20\%$: красный

Plateau Detection:

- Красная карточка при обнаружении

Protein Forecast:

- High urgency: красная рамка
- Medium urgency: жёлтая рамка

- Low urgency: зелёная рамка

Recovery Risk:

- High: красный фон
- Medium: жёлтый фон
- Low: оранжевый фон

Patterns:

- Alert: красный фон
- Warning: жёлтый фон
- Info: серый фон

Заключение

AI аналитика в LoviEat представляет собой комплексную систему из 11 блоков, которые:

1. **Анализируют** данные за последние 30 дней
2. **Вычисляют** различные метрики питания
3. **Сравнивают** ожидаемые и фактические результаты
4. **Выявляют** паттерны и проблемы
5. **Предоставляют** персонализированные рекомендации

Система использует модульную архитектуру, где большинство блоков независимы, но несколько блоков агрегируют результаты других для формирования общей оценки.

Все расчёты выполняются на клиенте в браузере с использованием данных из Firebase, что обеспечивает быстродействие и приватность данных.